

UNDERSTANDING THE ACORN ECONET LEVEL 3 FILE SERVER STRUCTURE

To examine and understand the Acorn Level 3 Econet file structure, a special disc has been created with the WFSINITT utility using a blank ADFS "L" floppy disc with parameters of 160 cylinders and 16 sectors per track.

A Passwords file was created with users BeebMaster and Games, in addition to Syst which is automatically added to the Passwords file when created.

Mounting the disc in the file server, a number of files and directories have been created to demonstrate the working of the file server structure.

The following is a listing of the directory catalogue and examine for all the objects on the file server part of the disc:

```
>*.
$          (008)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. $             Lib. $

AFILE      LR/R      BeebMaster DL/      BFILE      /WR      Directory1 DL/
Games      D/        L3Dirs      WR/      Passwords  /

>*EX
$          (008)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. $             Lib. $

AFILE      00005555 00004444 000800  LR/R      08/08/10 0000C1
BeebMaster 00000000 00000000 000200  DL/      08/08/10 000081
BFILE      FEDCBA01 76543210 000746  /WR      08/08/10 000141
Directory1 00000000 00000000 000AF0  DL/      08/08/10 0000B1
Games      00000000 00000000 000200  D/       08/08/10 000091
L3Dirs     FFFFFFFF FFFFFFFF 000400  WR/      08/08/10 000151
Passwords  00000000 00000000 000100  /        08/08/10 0000A1

>*DIR BEEBMASTER
>*.
BeebMaster(000)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. BeebMaster      Lib. $

>*EX
BeebMaster(000)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. BeebMaster      Lib. $

>*DIR $
>*DIR DIRECTORY1
>*.
Directory1(002)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. Directory1      Lib. $

Dir1ABCD  WR/      Directory2 DL/
```

```

>*EX

Directory1(002)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. Directory1     Lib. $

Dir1ABCD   FFFFFFFF FFFFFFFF   002000   WR/      08/08/10 0000D1
Directory2 00000000 00000000   0014FE   DL/      08/08/10 000101

>*DIR DIRECTORY2
>*.

Directory2(003)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. Directory2     Lib. $

Dir2EFGH    LWR/WR

>*EX

Directory2(003)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. Directory2     Lib. $

Dir2EFGH    EEFF0011 AABCCDD   001823   LWR/WR   08/08/10 000121

>*DIR $
>*DIR GAMES
>*.

Games      (000)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. Games      Lib. $

>*EX

Games      (000)      Owner
Level3MasterDisc    Option 02 (Run)
Dir. Games      Lib. $

```

PARTITIONING THE DISC INTO AN ADFS AND AN ECONET AREA

A Level 3 File Server disc contains an ADFS portion at the beginning followed by the Econet partition which uses up the remaining sectors on the disc.

The Level 3 specification states that the ADFS part of the disc will always occupy a complete number of cylinders. The actual amount of space reserved to ADFS will depend on the existing contents of the disc prior to its initialisation as a file server disc. This is done using the Level 3 initialiser “WFSINIT”.

Warning

Versions of the disc initialiser “WFSINIT” in circulation often skip the disc compaction routine which was contained in the original. This means it is very important to compact the disc before initialisation as WFSINIT assumes that the space to be allocated to Econet begins with the first free sector in the ADFS free space map. If the free space is fragmented then ADFS data will be overwritten and the disc is unlikely to be useable as an ADFS disc after WFSINIT has been run.

It is not possible to re-size an Econet partition once a disc has been initialised and so it is advisable to reserve some space in the ADFS part of the disc before running WFSINIT. This can be achieved using the *CREATE command and the file created can be deleted after the file server disc initialisation has taken place to “reclaim” the reserved space.

Creating the Partition

WFSINIT resizes the ADFS partition of the disc by making the following calculations:

1. The sector number of the first free sector is read from the free space map. This number is taken as being the total number of sectors used on the disc.
2. This is multiplied by 256 to calculate the amount of used space on the disc.
3. The sum of 16384 (16384) is added on to this figure as space for the ADFS area. The result is used as a total for the size of the ADFS area of the disc. This will always be a minimum figure due to rounding up to complete cylinders later.
4. The start cylinder for the NFS partition is now calculated, beginning by adding the number 7 (being the number of sectors required for the ADFS structure (2 for free space map, 5 for root directory) to the number of sectors taken up by the current ADFS data (the figure reached at step 3 divided by 256).
5. The total number of sectors for ADFS is now known.
6. If the figure calculated at step 3 is not a multiple of 256, 1 is added to the ADFS sector count. (This should never happen!)
7. The ADFS sector count is divided by the number of sectors per cylinder to give the number of cylinders required by ADFS.
8. If this is NOT a round number, then 1 is added to ensure enough space is reserved for all the ADFS data.
9. The cylinder number resulting is the start cylinder for NFS, as the Econet partition always starts at the beginning of a new cylinder.

Therefore, in our Test Disc example, the disc has no ADFS contents apart from the free space map and the root directory, taking up a total of 7 sectors. There are 16 sectors per cylinder and so the calculation as above is as follows:

1. First free sector is 7.
2. $7 * 256 = 1,792$.
3. $1,792 + 16,384 = 18,176$. This is the total size of the ADFS area
4. $7 + (18,176 / 256) = 78$
5. The total number of sectors required for ADFS is 78.
6. The figure at 3 is a multiple of 256 so no action is taken
7. ADFS sectors of $78 / 16$ sectors per cylinder = 4 cylinders, 14 sectors
8. Add 1 to the ADFS cylinder count to prevent overwriting the 14 sectors on the 5th cylinder
9. ADFS requires 5 full cylinders, addressed as cylinders 0-4, therefore the Econet partition will begin on the sixth cylinder, addressed as cylinder 5.

Having made the calculation, WFSINIT re-writes the ADFS free space map, keeping the amount of used space the same, since no additional data is to be added to the ADFS partition, but adjusting the total size of the ADFS area of the disc so that the total ADFS size equals the number of cylinders calculated at step 8 multiplied by the number of sectors per cylinder.

WFSINIT then adds two pointers to the free space map giving the location of the first sector of the NFS partition and the location of the copy of the first NFS sector.

Although comments in WFSINIT refer to writing “sector 2 and sector 12” this is presumably an error as the copy is always written to the start of the next cylinder after the first Econet cylinder.

The pointers are held at bytes &F6-&F8 (246-248) of sector 0 and sector 1 of the disc. The actual disc address of the first NFS sector and its copy, held in these pointers, will always be 1 more than might be expected, as the first sector of each cylinder (sector 0) is always taken up by the NFS free space bitmap, which will be explained later.

Therefore, as the NFS area is beginning at cylinder 5 on our Test Disc, we would expect the pointer to the first NFS sector to be sector 1 of cylinder 5, or sector &000051 (81) and for the copy to be at sector 1 of cylinder 6, or sector &000061 (97).

This shows a dump of ADFS sectors 0 and 1 on the Test Disc:

```
ADFS SECTORS 0 & 1
000000 07 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0000A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0000B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0000C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0000D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0000E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0000F0 00 00 00 00 00 00 00 51 00 00 00 00 00 50 00 00 A8 .....Q.....P..(
000100 49 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 I.....
000110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000120 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000130 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000170 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000180 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000190 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0001A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0001B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0001C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0001D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0001E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0001F0 00 00 00 00 00 00 00 61 00 00 00 00 55 6F 02 03 74 .....a.....Uo..t
```

The highlighted sections are as follows:

<i>Location</i>	<i>Value</i>	<i>Meaning</i>
Sector 0, bytes 0-2	07 00 00	The first free sector, = &7
Sector 0, bytes &F6-&F8	51 00 00	Location of NFS sector 1 = &000051 = 81
Sector 1, bytes 0-2	49 00 00	The number of free sectors = &49 = 73
Sector 1, bytes &F6-&F8	61 00 00	Location of copy of NFS sector 1 = &000061 = 97

Essentially the NFS partition has now been created by updating the ADFS free space map. The network partition does not need to be formatted as such, as the file server sees it as a block of ADFS sectors, which already exist from the time the disc was first formatted.

Writing the Bit Maps

WFSINIT must, however, create the bit maps which are required by the file server to keep a record of the available space in the file server partition.

The first sector in each cylinder of the NFS partition is given over to a bit map of the sectors in that cylinder.

Each sector is allocated a single bit, which may be set or clear. If the bit is set (ie. holds the value “1” then this indicates that the sector to which this bit relates is empty. If the bit is clear (ie. holds the value “0”) then this indicates that the sector to which this bit relates is occupied.

This bit map is kept in the first sector of each cylinder and is in the format

Byte 0 Bit map for the first 8 sectors, highest numbered sector to lowest numbered sector
Byte 1 Bit map for the second 8 sectors, highest numbered sector to lowest numbered sector

etc.

This system can only cope with a maximum of 2,048 sectors per cylinder (8 bits x 256 bytes = 2,048) as WFSINIT specifies a bitmap length of a single sector of 256 bytes.

This would normally be more than enough for floppy discs and Winchester Discs but could become a problem when using non-mechanical media such as compact flash memory cards where the “drive geometry” has to be artificially specified as concepts such as cylinders, heads and sectors per track do not have any meaning.

It is also the case that, for floppy discs, a good proportion of the available space in the file server area is wasted as 160 sectors (one sector for each track on the disc) is reserved to the bit map, and the bit map itself takes up only 2 bytes (1 bit per sector, 16 sectors per track under floppy ADFS = 16 bits = 2 bytes) of its 256 byte sector.

Under our Test Disc example, contents of sector &50 (being sector 0 of cylinder 5) would contain the bit map for that cylinder. As there are 16 sectors per cylinder on the disc, only 16 bits are required, taking up two bytes.

We know that sectors 0 and 1 of cylinder 5 are occupied – sector 0 is the bit map and sector 1 is NFS Sector 1, so we would expect the bit map for cylinder 5 to show that these two sectors are occupied and that sectors 3-15 are free.

The bit map should therefore be as follows:

Byte 0

Sector No	Sector 7	Sector 6	Sector 5	Sector 4	Sector 3	Sector 2	Sector 1	Sector 0
Value	1	1	1	1	1	1	0	0

Byte 1

Sector No	Sector 15	Sector 14	Sector 13	Sector 12	Sector 11	Sector 10	Sector 9	Sector 8
Value	1	1	1	1	1	1	1	1

Value held in byte 0 = $128+64+32+16+8+4$ =252 =&FC

Value held in byte 1 = $128+64+32+16+8+4+2+1$ =255 =&FF

This shows the bit map held at sector &50 of the Test Disc:

```
NFS SECTOR 0
005000 FC FF 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....
005010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
005090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

Writing the First Sector and Copy

WFSINIT writes the first sector, which contains the network partition identifier and other important information. This is written at sector 1 of the first cylinder available to the NFS partition. An identical copy of this sector is written at sector 1 of the second cylinder available to the NFS partition.

Creating the Root Directory and Passwords File

WFSINIT next creates a root directory for the Econet part of the disc. This is completely different to the ADFS directory structure and is discussed later on.

If a password file is requested, WFSINIT generates this, incorporating any users which are input. Directories within the root are also created for any user names.

Having written the root directory and passwords file to the disc, it is now initialised and may be used in the file server.

FIRST NFS SECTOR

Although known as the first NFS sector, this information is actually located as the second sector on the network partition because the first sector (sector 0) is always used for the cylinder free space bit map.

Only the first three dozen bytes or so of this sector are actually used, and contain the following data:

<i>Bytes</i>	<i>Description</i>
0-3	"AFS0", identifier for a file server disc
4-19	Disc name of up to 16 characters, padded with spaces
20-21	Number of cylinders on the disc, LSB followed by MSB
22-24	Number of sectors on the first, LSB first
25	Number of physical discs in the file server, generally 1
26-27	Number of sectors per cylinder, LSB first
28	Number of sectors per bit map, 1 according to WFSINIT

29	Next drive increment (specified as “Next drive” number entered in WFSINIT)
30	Unused
31-33	System Internal Name of Root Directory (see later)
34-35	Initialisation date, as entered in WFSINIT
36-37	First free cylinder on disc, LSB first
38	Floppy or Winchester Disc flag (0=Winchester disc) (N.B. This byte is not referred to in Acorn Application Note 75)

This is the first NFS Sector from our Test Disc:

0050100	41 46 53 30 4C 65 76 65 6C 33 4D 61 73 74 65 72	AFS0Level3Master
0050110	44 69 73 63 A0 00 00 0A 00 01 10 00 01 FF 01 71	Discq
0050120	00 00 28 D8 05 00 00 00 00 00 00 00 00 00 00 00	..(X.....
0050130	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0050140	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0050150	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0050160	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0050170	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0050180	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0050190	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00501A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00501B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00501C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00501D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00501E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00501F0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

The highlighted sections are as follows:

Bytes	Value	Meaning
0-3	41 46 53 30	“AFS0”, file server disc identifier
4-19	4C 65 76 65	
	6C 33 4D 61	
	73 74 65 72	
	44 69 73 63	
20-21	A0 00	“Level3MasterDisc”, disc name
22-24	00 0A 00	Cylinders on disc = &00A0 = 160
25	01	Sectors on disc = &000A00 = 2,560
26-27	10 00	Number of physical discs in file server = 1
28	01	Number of sectors per cylinder = &0010 = 16
29	FF	Number of sectors per bit map = 1
30	01	Next drive
31-33	71 00 00	Unused
34-35	28 D8	System Internal Name of Root directory = &000071
36-37	05 00	Date of initialisation = 8/8/2010
38	00	First free cylinder on disc = &0005
		Winchester disc identifier

N.B. WFSINIT appears to mark all discs as Winchester discs, if the value of byte 38 is to be relied upon as a floppy/Winchester identifier. This may be due to user amendments to the WFSINIT utility to initialise floppy discs when it was probably originally intended for Winchester discs only (hence “W”FSInit)

The copy of the first sector stored at sector &61 is identical:

006100	41 46 53 30 4C 65 76 65 6C 33 4D 61 73 74 65 72	AFS0Level3Master
--------	---	------------------

006110	44	69	73	63	A0	00	00	0A	00	01	10	00	01	FF	01	71	Disc	q
006120	00	00	28	D8	05	00	00	00	00	00	00	00	00	00	00	00	..(X.....	
006130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
006140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
006150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
006160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
006170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
006180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
006190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0061A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0061B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0061C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0061D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0061E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
0061F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		

SYSTEM INTERNAL NAME

Each object on the file server portion of the disc is given a System Internal Name. This is in fact a disc address pointing to the sector where the file can be found. The sector number given is not, however, the start of the file, but its Map Sector.

Files under Econet do not have to be contiguous, and so to provide a full list of the sectors occupied by a file, each file is given a Map Sector showing where the sectors which make up the file or object are located.

The System Internal Name (SIN) points to the Map Sector for the given file, and the Map Sector details the location of the fragments of the file itself.

The root directory has its SIN stored in NFS Sector 1 so that the file server knows where to find the root directory. All other files and directories on the disc have their SIN stored in the directory in which they are held, in the same way that ADFS contains the start sector for each of the objects in a directory.

Using our Test Disc example, we can see that the SIN of the test disc's root directory is &000071, in other words sector &71 of the disc, or sector 1 of cylinder 7. Remember that this location is where the root directory's Map Sector is stored, and NOT the root directory itself.

MAP SECTOR

As noted above, each object in the file server partition of the disc is given a map sector so that the file server can retrieve all of the sectors which make up an individual file. These may not be a continuous run of sectors, as would be the case in DFS or ADFS, so the file server must be told which sectors are stored where.

The map sector for a file is located at the disc address referred to in the file's System Internal Name and takes the following format:

Bytes	Meaning
0-5	"JesMap", identifier for a map sector
6	Map sequence number (see later)
7	Unused
8	LSB of object size. This is the number of bytes used in the last sector for the file, as the actual file length is not stored explicitly, but must be calculated.
9	Unused
10-12	Sector number of first group of sectors containing the file, LSB first

13-14 Number of sectors for first group, LSB first
15-17 Sector number of second group of sectors containing the file, LSB first
18-19 Number of sectors for second group, LSB first
etc.
255 Map sequence number. This must be identical to byte 6 or an FS Error 42 Broken Directory error will result.
The map for a file may be complex enough to require more than one sector and so the map sequence number identifies the number of in the sequence of map sectors.

We know from the contents of NFS Sector 1 that the Root Directory SIN is &000071 and so at Sector &71 we find the map sector for the root:

007100	4A	65	73	4D	61	70	00	00	00	00	72	00	00	02	00	00	JesMap.....r.....
007110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
007190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0071A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0071B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0071C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0071D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0071E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0071F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

<i>Bytes</i>	<i>Value</i>	<i>Meaning</i>
0-5	4A 65 73 4D 61 70	“JesMap”, map sector identifier
6	00	Copy of map chain sequence number
8	00	LSB of file length, in this case 0 bytes
10-12	72 00 00	Start sector of first group of sectors, &72
13-14	02 00	Number of sectors for first group, 2
15-17	00 00 00	Start sector of second group of sectors, 0
255	00	Map chain sequence number, 0

From the information given in the object’s map sector, we can discover the location of all the sectors making up the object and also calculate the size of the object.

The object starts at sector &72 and runs for 2 sectors. The entry for the second group of sectors relating to the file (bytes 15-17) is blank so there are no further blocks of sectors associated with this file. Therefore the total space used by the file is two sectors.

Byte 8 is zero indicating that the file is an exact number of sectors long. Thus the file length is &200 (512) bytes.

The map chain sequence number is zero indicating that there are no further map sectors associated with this file.

To retrieve the object, all we need to do is extract two sectors of data beginning at sector &72.

DIRECTORY STRUCTURE

Directories under Level 3 Econet may hold anything up to 255 entries but the space allocated to the directory depends upon information sent to the file server when creating a new directory. In addition, WFSINIT specifies a directory size of 2 sectors, sufficient for 19 entries, which is the default size of a directory used by the file server if no other information is given.

If the “number of entries” variable is sent to the file server when creating a directory then the directory will be created of sufficient size to accommodate this number of entries.

A directory will expand in size to accommodate a maximum of 255 entries as and when new objects are added to the directory.

It therefore follows that, unlike under ADFS, a directory need not be a fixed length, need not be contiguous and need not be an exact multiple of sectors in length. We will also see that the layout of a directory is also completely different that the structure under ADFS.

Calculating the Size of Directory

A directory will be of the following length:

$$18 + (\text{number of entries} \times 26)$$

Therefore, a directory with the default size of 19 entries is 512 bytes long, being $18 + (19 \times 26)$

A directory with the maximum number of 255 entries is 6648 bytes long, being $18 + (255 \times 26)$. This size is not an exact number of sectors, being 25 sectors and 248 bytes long.

Directory Layout

A file server directory contains a header of 17 bytes, followed by object information of 26 bytes per object, and a final byte being a copy of the master sequence number found in the header.

Not all of the entries may be occupied with object information and so a distinction is drawn between actual directory entries and free entries in the directory. The header information contains two pointers – one to the first actual directory entry and one to the first available free entry. Each of these entries contains a pointer to the next entry.

These pointers are calculated as an offset from the start of the directory, not as explicit disc sector addresses, so the directory object must be considered as a contiguous group of sectors beginning at zero when understanding the pointers.

Header

The first 17 bytes in the directory form a header as follows:

<i>Bytes</i>	<i>Meaning</i>
0-1	Pointer to information about first entry in the directory, LSB first
2	Directory Master Sequence Number
3-12	Directory name, padded with spaces
13-14	Pointer to first free entry in the directory, LSB first
15-16	Number of entries in the directory, LSB first.
N.B. The file server does not provide for directories with more than 255 entries and so the top byte should always be zero.	

Entries

Bytes	Offset	Meaning
--------------	---------------	----------------

17-18	0-1	Pointer to next entry in list, given as byte & sector from the start of the directory object If the entry is a directory entry then this pointer will point to the next directory entry If the entry is a free entry then this pointer will point to the next free entry and the remaining information in the entry will not be valid If the pointer is two zero bytes then there are no further entries in either the directory object information or the free entries information
19-28	2-11	Directory name padded with spaces
29-32	12-15	Load address of object, LSB first
33-36	16-19	Execution address of object, LSB first
37	20	Access Byte (see below)
38-39	21-22	Date object created or updated
40-42	23-25	System Internal Name of object
43-	0-	Repeated in blocks of 26 bytes for the total number of entries in the directory

Final Byte

Copy of the Master Sequence Number held at byte 2 of the directory. If the two values do not match, a Broken directory error will be reported and the directory will be inaccessible.

Essentially the two sets of pointers in the directory structure form two separate lists, a list of all the object entries in the directory, with each entry pointing to the location of the next entry, and a second list of all the free entries in the directory, again with each entry pointing to the location of the next free entry.

The size of the object is not stored in the object's directory information. This must be calculated using the object's map sector as discussed above.

OBJECT ACCESS

Principles of Object Access

Objects have owner and public access attributes. The access rights to a file may only be altered by a user with owner access which is determined by the user's privilege and user root directory. The separator "/" appears in the access string to separate owner attributes from public attributes.

Owner attributes may be Write-Access and Read-Access.

Public attributes may be Write-Access and Read-Access.

Write access allows a file to be updated or overwritten.

Read access allows a file to be loaded or its contents displayed.

Locked access prevents the file being deleted or overwritten.

The Locked attribute appears in the owner access section of the access string as there is no concept of public locked files, since all files are locked against deletion by a user with public access.

Directory objects do not permit the setting of owner or public write- and read-access. The only attribute which may be altered for a directory is the locked status. All directories have read access for all users but only write access for owners.

It is not possible to set a file to have owner write access only.

A file with public read access can be read by a user with owner access even if there is no owner read access.

It is possible to have files with public write access but no public read access. This may be used in message systems where users can add to a particular file due to their write access, but only the user with owner access to the file can read the file so that public users cannot read parts of the file added by other public users.

Access Byte

Byte 20 of each object entry in the directory is its access byte. The meaning of the bits is as follows:

Bit No.	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Bit Value	128	64	32	16	8	4	2	1
Meaning if set	Unused	Unused	Directory	Locked	Owner Write	Owner Read	Public Write	Public Read

N.B. The table above is completely at odds with the information given in Acorn Application Note 75. However, the above appears to be correct when verified against the access bytes for various files on the Test Disc. The Acorn application note does not mention a bit signifying a directory and there is no information in the note to explain how a directory is identified. Directories have a System Internal Name and Map sector just as files do and there does not appear to be any method of identifying a directory unless this is specified in the access byte as above.

Example Access Strings and Their Access Byte

<i>Access</i>	<i>Meaning</i>	<i>Bits in Access Byte</i>	<i>Value</i>
/	No read or write access	00000000	0
L/	Locked	00010000	16
LR/	Locked; owner read access	00010100	20
LWR/	Locked; owner read & write access	00011100	28
WR/	Owner read & write access	00001100	12
R/	Owner read access	00000100	4
L/WR	Locked; public read & write access	00010011	19
L/W	Locked; public write access	00010010	18
L/R	Locked; public read access	00010001	17
LR/WR	Locked; owner read; public read/write access	00010111	23
LR/W	Locked; owner read; public write access	00010110	22
LR/R	Locked; owner read; public read access	00010101	21
LWR/WR	Locked; owner read/write; public read/write	00011111	31
LWR/W	Locked; owner read/write; public write access	00011110	30
LWR/R	Locked; owner read/write; public read access	00011101	29
WR/WR	Owner read/write; public read/write access	00001111	15
WR/W	Owner read/write; public write access	00001110	14
WR/R	Owner read/write; public read access	00001101	13
R/WR	Owner read; public read/write access	00000111	7
R/W	Owner read; public write access	00000110	6
R/R	Owner read; public read access	00000101	5
/WR	Public read/write access	00000011	3
/W	Public write access	00000010	2
/R	Public read access	00000001	1

D/	Directory	00100000	32
DL/	Directory; Locked	00110000	48

N.B. As noted above, for files which have public read or write access with no corresponding owner read or write access, owner access is implied on the same basis as public access.

Setting an access string will increment the Master Sequence Number of the directory in which the object is held, but its date will not be altered. The Master Sequence Number is not recorded against individual objects, as it would be under ADFS.

Reference Table of Access Bytes

Use the following table as a ready reference for decrypting the access attributes for a file where the value of the access byte is known:

Access Byte (Decimal)	Access Byte (Hexadecimal)	Access String
0	&0	/
1	&1	/R
2	&2	/W
3	&3	/WR
4	&4	R/
5	&5	R/R
6	&6	R/W
7	&7	R/WR
12	&C	WR/
13	&D	WR/R
14	&E	WR/W
15	&F	WR/WR
16	&10	L/
17	&11	L/R
18	&12	L/W
19	&13	L/WR
20	&14	LR/
21	&15	LR/R
22	&16	LR/W
23	&17	LR/WR
28	&1C	LWR/
29	&1D	LWR/R
30	&1E	LWR/W
31	&1F	LWR/WR
32	&20	D/
48	&30	DL/

Test Disc Root Directory

We saw earlier that the root directory is two sectors in length starting at sector &72:

107200	7D 01 08	24 20 20 20 20 20 20 20 20 20 20 20	2F 01 07	}..\$	/..
107210	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
107220	00 00 00	00 00 00 00 00 00 00 00 00 00 11 00	00 00 00		
107230	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
107240	00 00 00	00 00 2B 00 00 00 00 00 00 00 00 00	00 00 00	+	
107250	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 45	E	
107260	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
107270	00 00 00	00 00 00 00 00 00 00 00 5F 00 00 00	00 00 00	_	
107280	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
107290	00 00 00	79 00 00 00 00 00 00 00 00 00 00 00	00 00 00	...y.....	
1072A0	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	93 00 00		
1072B0	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
1072C0	00 00 00	00 00 00 00 00 00 AD 00 00 00 00 00	00 00 00	-	
1072D0	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
1072E0	00 C7 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00	.G.....	
1072F0	00 00 00	00 00 00 00 00 00 00 00 00 00 E1 00	00 00 00	a....	
107300	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
107310	00 00 00	00 00 FB 00 00 00 00 00 00 00 00 00	00 00 00	{.....	
107320	00 00 00	00 00 00 00 00 00 00 00 00 00 00 00	15 00 00		
107330	01 00 00	00 00 00 00 00 00 00 00 00 00 00 00	00 00 00		
107340	00 00 00	00 00 00 00 00 00 00 00 B1 01 4C 33	44 69 72	1.L3Dir	
107350	73 20 20	20 20 FF FF FF FF FF FF FF FF 0C 28	D8 s	(X	
107360	51 01 00	97 01 42 46 49 4C 45 20 20 20 20 20	01 Q....BFILE	.	
107370	BA DC FE	10 32 54 76 03 28 D8 41 01 00 E5 01	41 :~.2Tv.(XA..e.A		
107380	46 49 4C	45 20 20 20 20 20 55 55 00 00 44 44	00 FILE UU..DD.		
107390	00 15 28	D8 C1 00 00 CB 01 44 69 72 65 63 74	6F ..(XA..K.Directo		
1073A0	72 79 31	00 00 00 00 00 00 00 00 30 28 D8 B1	00 ry1.....0(X1.		
1073B0	00 00 00	50 61 73 73 77 6F 72 64 73 20 00 00	00 ...Passwords ...		
1073C0	00 00 00	00 00 00 28 D8 A1 00 00 49 01 47 61	6D(X!..I.Gam		
1073D0	65 73 20	20 20 20 20 00 00 00 00 00 00 00 20	es		
1073E0	28 D8 91	00 00 63 01 42 65 65 62 4D 61 73 74	65 (X...c.BeebMaste		
1073F0	72 00 00	00 00 00 00 00 00 30 28 D8 81 00 00	08 r.....0(X....		

Bytes	Value	Meaning
0-1	7D 01	Pointer to first entry relating to an object = byte &017D of directory
2	08	Master Sequence Number = 8
3-12	24 20 20 20 20 20 20 20 20 20 20	Directory name padded with spaces = "\$"
13-14	2F 01	Pointer to first entry which is free = byte &012F of directory
15-16	07 00	Number of entries in directory = 7

The 26-byte block relating to the first directory entry begins at &017D as is as follows:

Bytes	Offset	Value	Meaning
&17D-&17E	0-1	E5 01	Pointer to next (ie. second) object entry in directory
&17F-&188	2-11	41 46 49 4C 45 20 20 20 20 20	"AFILE", filename
&189-&18C	12-15	55 55 00 00	Load address = &00005555
&18D-&190	16-19	44 44 00 00	Execution address = &00004444
&191	20	15	Access byte = &15 = LR/R
&192-&193	21-22	28 D8	Date
&194-&196	23-25	C1 00 00	System Internal Name of file, ie. Sector &C1

The pointer shows us that the next entry is BeebMaster, whose object information begins at byte &1E5 of the directory. The load address, execution address, access byte, date and system internal number all appear in the directory examine reproduced at the beginning of this document.

The pointers for the list of objects held in the directory are arranged so that the entries are referred to in alphabetical order, although the entries themselves are not stored in the directory alphabetically. It appears that the file server fills up the directory from the end. The first entries created in the root directory were the directories BeebMaster and Games, and then the Passwords file, which are, in reverse order, the last three entries in the directory.

We can follow the chain of objects contained within the directory by following the directory object pointers:

7D 01	Pointer to the first directory object contained at bytes 0-1 of the directory
E5 01	Pointer to the second directory object, held with the entry for the first object "AFILE"
63 01	Pointer to the third directory object, held with the entry for the second object "BeebMaster"
97 01	Pointer to the fourth directory object, held with the entry for the third object "BFILE"
CB 01	Pointer to the fifth directory object, held with the entry for the fourth object "Directory1"
49 01	Pointer to the sixth directory object, held with the entry for the fifth object "Games"
B1 01	Pointer to the seventh directory object, held with the entry for the sixth object "L3Dirs"
00 00	Null bytes indicating no further entries, held with the entry for the seventh object "Passwords"

As with the object entries, the free entries begin towards the end of the directory layout and work backwards to the beginning. We can follow the chain of free entries in the directory by following the free entry pointers:

2F 01	Pointer to the first free entry contained at bytes 13-14 of the directory
15 01	Pointer to the second free entry, held at the first free entry
FB 00	Pointer to the third free entry, held at the second free entry
E1 00	Pointer to the fourth free entry, held at the third free entry
C7 00	Pointer to the fifth free entry, held at the fourth free entry
AD 00	Pointer to the sixth free entry, held at the fifth free entry
93 00	Pointer to the seventh free entry, held at the sixth free entry
79 00	Pointer to the eighth free entry, held at the seventh free entry
5F 00	Pointer to the ninth free entry, held at the eighth free entry
45 00	Pointer to the tenth free entry, held at the ninth free entry
2B 00	Pointer to the eleventh free entry, held at the tenth free entry
11 00	Pointer to the twelfth free entry, held at the eleventh free entry
00 00	Null bytes indicating no further free entries, held at the twelfth free entry

Thus there are 7 objects and 12 free entries in the root directory, make up the total number of 19 entries for the root as created by "WFSINIT".

Only the first two bytes, being the pointer to the next entry, are used in the free entries although of course 26 bytes are allocated to each entry as these bytes are used once the entry becomes filled with information relating to an object.

PASSWORDS FILE

The Passwords file under the Level 3 File Server contains all the users on the disc, their passwords, the amount of space allocated to the user, the user boot option and user privilege.

The passwords file is stored on the disc as \$.Passwords and is set with no access by WFSINIT on creation if requested when initialising the disc. The file length must be a complete number of sectors and a passwords file with a length of 1 sector, ie. 256 bytes (enough space for 19 users) is created by WFSINIT and the file may be extended if this size becomes insufficient. There does not appear to be

an upper limit on the size of the password file, although a very large number of users will make the file server less efficient.

The passwords file is arguably the most important file on the file server disc, as the file server cannot function without a valid passwords file on one of the discs mounted by the file server when it is started.

If the file server fails to find a valid passwords file when it is started, it will not start.

If the passwords file in use by the file server is deleted or is edited to a length not equal to a complete number of sectors, then the file server will stop the next time the passwords file is accessed. The passwords file is accessed frequently, whenever a user logs off, attempts to log on, changes his password or boot option or saves any data.

Despite having no read or write access, the passwords file is not locked against deletion and may therefore be deleted (possibly accidentally) easily by a System user without any warning being issued.

Password File Layout

The information relating to each user is 31 bytes long. There is no header information or identifier at the beginning of the password file, it is simply a set of 31-byte blocks of data. The remaining bytes between the end of the last block of user data and a full sector are unused.

The 31-byte data block for each user takes the following form:

Bytes	Meaning
0-19	EITHER: i) up to 10 characters of user group identifier, terminated with a full stop (“.”) if the user group identifier is less than 10 characters, and up to 10 characters of user identifier terminated with a carriage return (&D) if less than 10 characters; OR ii) up to 10 characters of user identifier only (no group identifier) terminated with a carriage return if less than 10 characters
20-25	Up to 6 characters of user password, terminated with a carriage return if less than 6
26-29	Bytes of free space remaining to the user, LSB first
30	Status Byte (see below)

User Status Byte

The table below shows the meaning of the bits in the status byte associated with each user in the passwords file:

Bit No.	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Bit Value	128	64	32	16	8	4	2	1
Meaning if set	In use (if clear the data block will be ignored)	System Privileged User	Unused	Unused	Unused	Unused	Boot Option: 0 = off 1 = load 2 = run 3 = execute	

User Space Accounting

The Level 3 file server introduces space accounting for users. Each user is allocated a set number of bytes of storage space when users are entered during the running of “WFSINIT” or when users are

subsequently created on the file server disc.

The number of bytes of storage remaining to each user is maintained in the passwords file, and the total is debited and credited upon every action by the user which causes more or fewer bytes of space to be taken up; for example saving a file will debit the user free space amount by the length of the file and deleting a file will credit the file's length to the user.

Attempting to use space exceeding the amount remaining to the user will result in an Insufficient space error and the operation will not be completed.

WFSINIT allocates free space of 263,172 bytes to the SYST user and other users entered when generating the passwords file; the same figure is allocated by the file server when users are subsequently created by the file server.

This amount of space can soon be used up, especially when copying up files to a newly-initialised file server disc, and considerable inconvenience may be caused during a long copying process which aborts unexpectedly due to an "Insufficient space" error. It is recommended that the amount of user free space is increased prior to any large-scale copying operation.

The file server does not accord any relationship to the amount of free space available to an individual user, the total amount of space allocated to all users in the file server and the actual amount of disc space available. Therefore, it is perfectly possible for a user, or more than one user, to be allocated a number of bytes of space which exceeds the amount of space available on the disc. Doing so would effectively give such users an "unlimited" amount of space.

System Privileged Users

Special privileges are accorded to System users. They have owner access to all files and directories in the file server and are the only users permitted to create and delete users and to set the amount of free space allocated to users.

There is, however, no special privilege accorded to System users in terms of user free space: they are subject to the same space accounting rules as all other users.

When requested to generate a Passwords file, WFSINIT will create a user called SYST with System privilege.

There is no limit to the number of System users in the file server, nor is there any requirement for a System user to be called SYST, although many Econet utilities will assume that the user SYST exists and has System privilege.

There is no requirement for any System privileged users to exist on the file server. However, managing the network without System privilege will be considerably more difficult, as System users cannot be created if none exist, and certain functions will consequently be unavailable.

As a security measure, the main disc or discs in use in the file server need not contain any System users, but a special disc with a System user could be held in reserve for use when System privileged access is required. However, this would necessitate stopping and re-starting the file server to introduce the special disc.

Test Disc Passwords File

10A200	53 79 73 74 0D 00 00 00 00 00 00 00 00 00 00 00	Syst.....
10A210	00 00 00 00 0D 00 00 00 00 00 00 9C 88 FF 3F C2 42	?BB
10A220	6F 6F 74 0D 00 00 00 00 00 00 00 00 00 00 00 00	oot.....
10A230	00 00 00 0D 00 00 00 00 00 00 F4 FF FF 3F 80 4C 69	t..?.Li
10A240	62 72 61 72 79 0D 00 00 00 00 00 00 00 00 00 00	brary.....
10A250	00 00 0D 00 00 00 00 00 00 00 00 00 40 80 4C 69 62	@.Lib
10A260	72 61 72 79 31 0D 00 00 00 00 00 00 00 00 00 00	rary1.....
10A270	00 0D 00 00 00 00 00 00 00 00 00 40 80 55 74 69 6C	@.Util
10A280	73 0D 00 00 00 00 00 00 00 00 00 00 00 00 00 00	s.....
10A290	0D 00 00 00 00 00 00 00 00 00 40 80 42 65 65 62 4D	@.BeebM
10A2A0	61 73 74 65 72 0D 00 00 00 00 00 00 00 00 00 0D	aster.....
10A2B0	00 00 00 00 00 00 00 00 00 40 80 47 61 6D 65 73 0D	@.Games.
10A2C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 0D 00	
10A2D0	00 00 00 00 00 00 00 00 40 80 00 00 00 00 00 00	@.....
10A2E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
10A2F0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Bytes	Value	Meaning
0-19	53 79 73 74 0D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	User ID - "Syst" followed by a carriage return. The remaining bytes are not used as there is no group identifier as part of the user name.
20-25	0D 00 00 00 00 00	Password, blank so the string begins with carriage return
26-29	9C 88 FF 3F	User free space remainig = &3FFF889C = 1,073,711,260 bytes
30	C2	User Status Byte = &C2 = 11000010 = User ID in use (128), System Privilege (64), Boot option Run (2)

Bytes	Offset	Value	Meaning
63-82	0-19	4C 69 62 72 61 72 79 31 0D 00 00 00 00 00 00 00 00 00 00 00	User ID - "Library1" followed by a carriage return. The remaining bytes are not used as there is no group identifier as part of the user name.
83-88	20-25	0D 00 00 00 00 00	Password, blank so the string begins with carriage return
99-92	26-29	00 00 00 40	User free space remainig = &40000000 = 1,073,741,824 bytes
93	30	80	User Status Byte = &80 = 10000000 = User ID in use (128), Boot option Off (0)

Although only two users were entered during "WFSINIT" ("BeebMaster" & "Games"), it seems that additional users are also entered into the Passwords file: Syst, Boot, Library, Library1, and Utils.

The Passwords file for the Test Disc was created using a modified version of WFSINIT to allocate each user free space of &40000000 bytes to prevent the Insufficient space problem described above.